

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	S. Roshni	Reg. No.:	192525185
Section / Batch:	AI & ML	Date:	26/08/2026

Code of Conduct

I S. Roshni (192525185)

(Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Roshni

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

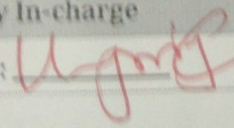
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1) Linear Probing vs Separate Chaining Collision Handling in Hashing

Parameters	Linear Probing	Separate Chaining
Clustering effect	Suffers from primary clustering because consecutive empty positions are searched	very less clustering because collided elements are stored in a linked list/bucket
Memory Usage	Uses less memory because elements are stored inside the hash table itself	Uses extra memory linked lists/pointers
High load factor	performance decreases rapidly as load factor increases	Handles high load factor better than linear probing
Search	Average: $O(1)$, worst: $O(n)$	Average: $O(1)$, worst: $O(n)$
Deletion	more complicated because deleted position need special handling	Easy; simply remove the element from chain
Advantages	Simple, Cache-friendly & memory efficient	Flexible & Performs better at high load factors

2) Quick Sort vs Merge Sort for 10 Million
Linked-List Nodes

Parameter	Quick Sort	Merge Sort
Time - Best	$O(n \log n)$	$O(n \log n)$
Time - Average	$O(n \log n)$	$O(n \log n)$
Time - worst	$O(n^2)$	$O(n \log n)$
Extra space for linked list	$O(\log n)$ average recursion stack; can become $O(n)$.	$O(\log n)$ recursion stack; merging can be performed efficiently
In-Place	Usually considered in-place for arrays; linked-list implementation can be more complex.	Not in place in the usual array implementation, but linked lists can be merged
Cache behavior	Better cache locality for arrays, but less important for linked lists.	Sequential traversal is suitable for linked list
linked-list Operations	Partitioning is possible but pointer manipulation makes it less convenient	very efficient because splitting & merging linked lists are natural operations

3)

<u>Quadratic Probing Vs Double Hashing.</u>		
<u>Factor</u>	<u>Quadratic probing</u>	<u>Double Hashing</u>
Clustering	Secondary	Very low
Collision handling	Good	Better
Probe sequence	Quadratic intervals	Uses second hash function
Dense tables	Less suitable	More suitable
Search	Average $O(1)$	Average $O(1)$
Insertion	Average $O(1)$	Average $O(1)$
Advantage	Simple	Better distribution
Limitation	Secondary	More complex
memory	No extra linked list	No extra linked list
Best Choice	moderate load	Dense hash tables

Conclusion: Double Hashing is better for dense hash tables because it reduces clustering & Provides better collision resolution.

4) Best Case vs Worst Case Sorting

Factor	Best Case	Worst Case
Meaning	Minimum execution Time	Maximum execution time
Quick Sort	$O(n \log n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$
Practical Use	Shows ideal Performance	Shows max required time
Algorithm Selection	Useful for favorable input	Important for predictable Performance

Conclusion: —

Worst-case complexity is important when choosing an algorithm for large applications because it shows the max time required

Recursive Vs Iterative Quick sort

Factor	Recursive	Iterative
Implementation	Easy	More complex
Stack	uses system call stack	uses explicit stack
Average space	$O(\log n)$	$O(\log n)$
worst space	$O(n)$	$O(n)$
Stack overflow	Higher risk	lower risk
code	Shorter	Longer
Control	less control	more control
Performance	Good	Good
Advantage	Simple to understand	Avoids recursive calls
Best use	Small/normal inputs	Large inputs

Conclusion: Recursive Quick sort is easier to implement, while iterative Quick sort is safer for large inputs because it avoids deep calls.